Solution: In-class assignment: Sorting part of an array

COSC 2230 (CS III) Fall 2026 | Augustana U. | Friday, November 6, 2026

Task 1: Trace

lo	mid	hi	a[0]	a[1]	a[2]	a[3]	a[4]	a[5]
1	2	3	it	the	was	best	of	times
0	1	3	it	the	was	best	of	times
4	5	6	it	the	was	best	of	times
3	4	6	it	the	was	best	of	times
0	3	6	best	it	of	the	times	was

- `sort(a, 0, 6)` splits at mid = 3. The left half `sort(a, 0, 3)` has 3 entries and splits at mid = 0 + 3/2 = 1: `a[0, 1)` has one entry (base case), so the first merge is the one of `a[1, 3)`.
- The whole left half is sorted before the right half is touched.
- Only two merges change the array: `was the` becomes `the was`, and the last merge. In the other three, the two halves are already in order.

Task 2: Find the bug

The program crashes in `merge()` with a `NullPointerException`:

```
Cannot store to object array because "Merge.aux" is null
```

`aux` is a field that is never given an array at its declaration, so it starts as `null`. Only `sort(a)` creates the array. This program calls `sort(a, 1, 5)` directly (three arguments: Java picks that method), so `aux` is still `null` when the first `merge()` writes `aux[k]`.

Task 3: Fix it

```
public static void sort(String[] a)
{
    sort(a, 0, a.length);
}

public static void sort(String[] a, int lo, int hi)
{
    aux = new String[hi - lo];
    sortRange(a, lo, hi);
}

private static void sortRange(String[] a, int lo, int hi)
```

```
{
    int N = hi - lo;
    if (N <= 1) return;
    int mid = lo + N/2;
    sortRange(a, lo, mid);
    sortRange(a, mid, hi);
    merge(a, lo, mid, hi);
}
```

- The public `sort(a, lo, hi)` creates `aux`, then starts the recursion. Every way into the sort now passes through that line.
- The recursion moves into the private helper `sortRange()`. It needs a new name: two methods called `sort` with the same parameters are not allowed. Its recursive calls must call `sortRange()`, not `sort()`: `sort()` would create a new, smaller `aux` in every call, and the merges further up would run past its end.
- `sort(a)` just sorts the range `a[0, a.length)`, so `aux` is created in one place only.

How big must `aux` be at least? `hi - lo` entries, the size of the range being sorted. `merge()` fills `aux` from index 0, and the largest merge, the last one, covers all of `a[lo, hi)`. `a.length` also works, but it is more than needed.

With the fix, `Merge.sort(a, 1, 5)` sorts only `a[1]` to `a[4]`:

```
it best of the was times
```

and `Merge.sort(a)` still gives `best it of the times was`.